Integrating Google Login (OAuth 2.0) into Your Web Project

This document provides a comprehensive, step-by-step guide to adding Google Login to a full-stack web application. We will cover the configuration in the Google Cloud Console, a frontend implementation using React, and backend token verification using Node.js and Express.

Phase 1: Google Cloud Console Configuration

Before writing any code, you need to register your application with Google to obtain your Client ID and Client Secret.

Step 1.1: Create a New Project

1. Go to the Google Cloud Console.
2. Click on the project drop-down menu at the top of the screen and select **New Project**.
3. Enter a project name (e.g., "My Auth Project") and click **Create**.

Step 1.2: Configure the OAuth Consent Screen

The consent screen is what users see when they are prompted to log in.

1. Navigate to **APIs & Services > OAuth consent screen**.
2. Choose **External** (if your app is for public use) and click **Create**.

Field	Description
App Name	The name of your application as it will appear to users.
User Support Email	Your email address for user support inquiries.
Authorized Domains	Your production top-level domain (can be left blank for localhost testing).
Developer Contact Information	An email address for Google to contact you regarding the project.

Save and continue through the *Scopes* and *Test Users* screens (add your own email as a test user if the app is still in testing mode).

Step 1.3: Generate OAuth 2.0 Client IDs

1. Navigate to **APIs & Services > Credentials**.
2. Click **+ Create Credentials** and select **OAuth client ID**.
3. Select **Web application** as the Application type.
4. Under **Authorized JavaScript origins**, add your frontend URL (e.g., http://localhost:3000).
5. Under **Authorized redirect URIs**, add your backend redirect URL or frontend callback URL if needed (e.g., http://localhost:3000).
6. Click **Create**. You will receive a **Client ID** and **Client Secret**. Save these securely.

Phase 2: Frontend Implementation (React)

We will use the @react-oauth/google library to handle the frontend login button and retrieve the Google identity token.

Step 2.1: Install Dependencies

```
npm install @react-oauth/google axios
```

Step 2.2: Wrap Application in GoogleOAuthProvider

In your entry file (e.g., index.js or main.jsx), wrap your App component:

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';
import { GoogleOAuthProvider } from '@react-oauth/google';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <GoogleOAuthProvider clientId="YOUR_GOOGLE_CLIENT_ID">
    <App />
  </GoogleOAuthProvider>
);
```

Step 2.3: Create the Login Component

Use the GoogleLogin component to render the button and capture the credential token upon success.

```
import React from 'react';
import { GoogleLogin } from '@react-oauth/google';
import axios from 'axios';

const Login = () => {
 const handleSuccess = async (credentialResponse) => {
  console.log("Encoded JWT ID token:", credentialResponse.credential);

  try {
   // Send the Google token to your backend for verification
   const res = await axios.post('http://localhost:5000/api/auth/google', {
    token: credentialResponse.credential,
   });
   console.log("Backend response:", res.data);
   // Save your application's JWT session token in localStorage or Context
   localStorage.setItem('authToken', res.data.token);
  } catch (error) {
   console.error("Authentication failed", error);
  }
 };

 return (
  <div>
   <h2>Sign in to your account</h2>
   <GoogleLogin
    onSuccess={handleSuccess}
```

```
        onError={() => {
          console.log('Login Failed');
        }}
      />
    </div>
  );
};

export default Login;
```

Phase 3: Backend Verification (Node.js & Express)

Never trust frontend authentication alone. You must send the Google identity token to your backend, verify it with Google's servers, and then issue your own application-specific JWT or session cookie.

Step 3.1: Install Dependencies

```
npm install express cors google-auth-library jsonwebtoken dotenv
```

Step 3.2: Set up the Verification Endpoint

Use the google-auth-library to securely verify the token signature and payload.

```
require('dotenv').config();
const express = require('express');
const cors = require('cors');
const { OAuth2Client } = require('google-auth-library');
const jwt = require('jsonwebtoken');

const app = express();
app.use(cors());
```

```
app.use(express.json());

const client = new OAuth2Client(process.env.GOOGLE_CLIENT_ID);

app.post('/api/auth/google', async (req, res) => {
  const { token } = req.body;

  try {
    // 1. Verify the token with Google
    const ticket = await client.verifyIdToken({
      idToken: token,
      audience: process.env.GOOGLE_CLIENT_ID,
    });

    // 2. Extract user data
    const payload = ticket.getPayload();
    const { sub: googleId, email, name, picture } = payload;

    // 3. Check if user exists in your Database (e.g., MongoDB, PostgreSQL)
    // let user = await User.findOne({ email });
    // if (!user) {
    //     user = await User.create({ googleId, email, name, picture });
    // }

    // 4. Generate your own application JWT
    const appToken = jwt.sign(
      { userId: googleId /* replace with your DB user._id */, email },
      process.env.JWT_SECRET,
      { expiresIn: '1d' }
    );

    res.status(200).json({
      message: 'Login successful',
```

```
    token: appToken,
    user: { name, email, picture }
  });

 } catch (error) {
  console.error("Token verification failed:", error);
  res.status(401).json({ message: 'Invalid Google Token' });
 }
});

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
```

Phase 4: Security Best Practices

- **Environment Variables:** Never hardcode your GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET, or JWT_SECRET in your source code. Always use a .env file.
- **Token Validation:** Always verify the token's audience on the backend (which should match your Client ID) to prevent confused deputy attacks.
- **Secure Session Management:** If storing your application's JWT on the frontend, consider setting it as an HttpOnly cookie rather than returning it in the JSON response to prevent Cross-Site Scripting (XSS) attacks from stealing the token.